

```

In [2]: import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import classification_report, accuracy_score, precision_score

import pennylane as qml
from pennylane.optimize import NesterovMomentumOptimizer

# Load dataset (adjust path if downloaded locally)
df = pd.read_csv("../Datasets For Classification/Diabetes/diabetes.csv")

# Features and Labels
X = df.drop(columns=["Outcome"]).values
y = df["Outcome"].values

# Preprocessing
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# For quantum simulation, reduce features (e.g., pick 4 most relevant)
X_scaled = X_scaled[:, :4]

X_train, X_test, y_train, y_test = train_test_split(X_scaled, y, test_size=0.2,

# Quantum circuit setup
n_qubits = 4
dev = qml.device("default.qubit", wires=n_qubits)

def variational_circuit(params, x):
    for i in range(n_qubits):
        qml.RY(x[i], wires=i)
    qml.templates.StronglyEntanglingLayers(params, wires=range(n_qubits))
    return qml.expval(qml.PauliZ(0))

@qml.qnode(dev)
def circuit(params, x):
    return variational_circuit(params, x)

def predict(params, X):
    preds = []
    for x in X:
        output = circuit(params, x)
        preds.append(1 if output >= 0 else 0)
    return np.array(preds)

def cost(params, X, y):
    preds = np.array([circuit(params, x) for x in X])
    return np.mean((preds - (2 * y - 1))**2)

# Initialize parameters
np.random.seed(42)
params = 0.01 * np.random.randn(3, n_qubits, 3)
optimizer = NesterovMomentumOptimizer(stepsize=0.5)

# Training Loop
epochs = 30
for epoch in range(epochs):

```

```

params = optimizer.step(lambda p: cost(p, X_train, y_train), params)
if epoch % 5 == 0:
    loss = cost(params, X_train, y_train)
    print(f"Epoch {epoch}: Loss = {loss:.4f}")

# Prediction
y_pred = predict(params, X_test)

# Evaluation
print("\nClassification Report:\n")
print(classification_report(y_test, y_pred))

metrics = {
    "accuracy": accuracy_score(y_test, y_pred),
    "precision": precision_score(y_test, y_pred),
    "recall": recall_score(y_test, y_pred),
    "f1_score": f1_score(y_test, y_pred)
}

# Print summary
print("\nSummary Metrics:")
for key, val in metrics.items():
    print(f"{key.capitalize()}: {val:.4f}")

```

c:\Users\Mr. Nitin\Desktop\quantum_ML\lib\site-packages\pennylane_grad.py:216: UserWarning: Attempted to differentiate a function with no trainable parameters. If this is unintended, please add trainable parameters via the 'requires_grad' attribute or 'argnum' keyword.

warnings.warn(

Epoch 0: Loss = 1.9414
 Epoch 5: Loss = 1.9414
 Epoch 10: Loss = 1.9414
 Epoch 15: Loss = 1.9414
 Epoch 20: Loss = 1.9414
 Epoch 25: Loss = 1.9414

Classification Report:

	precision	recall	f1-score	support
0	0.56	0.05	0.09	99
1	0.35	0.93	0.51	55
accuracy			0.36	154
macro avg	0.45	0.49	0.30	154
weighted avg	0.48	0.36	0.24	154

Summary Metrics:
 Accuracy: 0.3636
 Precision: 0.3517
 Recall: 0.9273
 F1_score: 0.5100